The Evolution of The Vibeathon: A Study of TreeHacks Repositories (2015–2026)

Ryan McCormick

***Abstract*—This paper tracks how TreeHacks software projects changed from 2015 to 2026, with special focus on the post-2022 generative AI era. Using metadata from Devpost submissions, including linked GitHub repositories with file information and commit histories, I analyze event participation, programming languages, project scope, development practices, and code security. The central result is that AI has changed the shape of how people work in hackathons. Recent projects are more likely to use Python and Typescript, have a larger project size/scope, have larger commits, and touch more files per commit. There's a similar growth in deletion activity, suggesting teams are often revising and iterating on generated code. The strongest negative feature that we see is a sharp rise in environment-variable exposure (especially `.env` leaks in commit history). Overall, the evidence suggests that AI lowered the barrier to making ambitious projects, but also created new risks around code review and secret security.**

I. Introduction

THE TREEHACKS EVENT at Stanford University is one of the largest and most prestigious student hackathons in the world, with the most recent event hosting 1,099 participants and $1,030,875 in prizes [1]. For those unfamiliar with the term, a hackathon is an event where teams of developers build open-ended projects (sometimes lead by a prompt) in a short time window, often 24–48 hours. The goal is to build and present a working prototype to an array of judges, and possibly win prizes.

Hackathons have always rewarded speed, ambition, and good demos that look good at first glance, but might be superficial. In that sense, they were always close to what many people now label "vibe-coding": a style of AI-assisted development where the focus is on quickly generating large chunks of code, often without deep knowledge of how it works, and sometimes without even looking at the generated code [2]. With the explosive growth of large language models since late 2022, the tools for vibe-coding have become widely accessible, and many developers have begun to adopt this style for rapid prototyping.

This paper asks a straightforward question: how did those new development tools change the observable behavior of hackathon teams? If vibe-coding is significantly affecting project development, we should see it in the history of repositories and commits, file changes, language choices, and general code safety.

The argument in this paper should be met with caution. This as an observational trend; it's not a proof. Still, the patterns are strong enough to support a reasonable claim: AI assistance appears to have expanded what teams can make in a weekend, while also increasing the prevalence of complexity and security mistakes.

II. Data and Method

A. Data Assembly

The analysis combines TreeHacks project listings (2015–2026) with their linked public GitHub repositories and metadata retrieved through the GitHub API [3]. Each project can contribute one or more repositories. For each repository, I aggregate:

- basic repository metadata (stars, forks, size, visibility, archive status),
- language-byte breakdowns and file inventories,
- commit-level statistics (additions, deletions, files touched, message text, timestamps),
- contributor and author summaries.

I report trends by event year and sometimes highlight a practical before/after split: 2015–2022 vs. 2023–2026, representing the public release of ChatGPT in November 2022 [4].

B. Behavioral Proxies

Because "vibe-coded" is not a standard label that people willingly tag their commits with, I use some proxies:

- large-commit rates (e.g., commits with >500 or >1000 additions),
- additions per commit and additions per file touched,
- single-commit / low-commit project prevalence,
- AI-related terms in commit messages,

For a very trivial security analysis, I specifically track `.env`-style files both in the current tree and in commit history, because presence in previous commits still persists even after files are later removed.

III. A Changing Environment: Participation and Language

The first result is simple: TreeHacks participation is much larger in recent cohorts (Fig. 1). This expansion matters because it raises the baseline diversity of the competition, such that later-year trends represent larger, broader shifts in the projects of the event, rather than a small number of outliers. You'll also notice the impact of the COVID-19 pandemic on the treehacks event in 2022, which was held virtually and had a much smaller number of projects with linked repositories [5].

Language composition also shifted. Python and TypeScript are substantially more prevalent in recent years than in the early JavaScript-heavy period (Figs. 2 and 3). This stack profile is consistent with the AI-era workflow: Python for model and data pipelines, TypeScript for user-facing interfaces.

IV. The Unit of Work is Growing

The core pattern is that project updates have increased in size. The aggregate dashboard (Fig. 4) shows that the footprints of commits have grown significantly in the past few years.

This growth is mirrored in Fig. 6, which shows growth in total files changed per project. Similarly, Fig. 7 shows that the scope of projects has expanded, requiring more files, more lines of code and, most interestingly, more work to build.

V. Business in the Front, Party in the Back

Deletions per commit remain substantial and rise in later years (Fig. 8), which might reflect a large quantity of big revisions to code after it is generated. If vibe-coding were very clean at this point and could one-shot a perfect project, then we wouldn't see this pattern. Instead, the deletions suggest that teams are still consistently having to review and refine the code that comes out of their tools.

We can get a deeper view on iteration by looking at commit messages, where we see a rise in terms like "fix" and "feat", and a decrease in terms like "update" and "test" (Fig. 9). This might represent a less active testing and polishing phase, instead a more feature-oriented, trial-and-error approach to building. We can also see patterns in Fig. 10 that support a similar model: fast front-loaded generation followed by grand adjustments. In other words, teams appear to generate first a bunch of code at first and then carve out solutions from that big chunk, rather than iteratively building up from a small base.

We can also see this pattern very clearly in the plot of commit size distributions relative to the proportion of the projects' lifetime (Fig. 11). The post-2023 projects show a much higher density of large commits in the early stages of the project, which supports the idea of front-loaded generation followed by later refinement. The pre-2023 projects show a more even distribution of commit sizes across the timeline, which might reflect a more traditional development process.

VI. AI Visibility in Commit Language

AI influence is not only guess-work on our part. It is increasingly explicitly stated in commit messages itself (Fig. 12), where AI-related terms appear very frequently. This means that teams are not only using AI tools, but also acknowledging their use in the commit history. It's a social shift, a sort of pride or identity rather than something hidden or shameful. It could also be a reference to the increase of AI-related prizes and tracks in the event.

VII. Security Basics: `.env` Exposure

Alas, the enemy of prompt-engineers: `.env` files. These are text files that often contain environment variables, which can include sensitive information like API keys, database credentials, and other secrets. The presence of `.env` files in repositories, especially in commit history, is a strong signal of poor code-management skills. It's very easy to prevent this from happening, but it seems programmers have been slacking off recently, or perhaps not caring, because the graphs show a sharp rise in exposure of `.env` files in recent years (Fig. 13b). Though, notably, a good amount of these exposed files appear to be example or template files, but still a non-negligible portion are secret-like files with actual credentials. Notably, I've managed to extract 20 `OPENAI_API_KEY`s, 7 `GEMINI_API_KEY`s, and 6 `ANTHROPIC_API_KEY`s from the commit history of these repositories, mostly from the 2023–2026 period.

VIII. Discussion and Implications

Three implications stand out.

First, AI-era tools appear to broaden project ambition in hackathon constraints. Teams can build up larger, more complex repositories with fewer manual steps.

Second, the conceptual shape of development may be changing. When larger chunks are generated and integrated rapidly, the mental burden shifts from writing code to verifying and understanding code. We're creating new bottlenecks around code iteration, rather than upfront design and incremental development.

Third, security and deployment hygiene are lacking. The `.env` trend suggests many teams are shipping quickly without even glancing at their files, maybe forgetting those secrets even exist in the first place.

The short version is that recent TreeHacks repositories are more ambitious, more complex, and more risky. But it's overwhelmingly clear that vibe-coding to some degree is now the norm of the hackathon; we're at the beginning of a new era and it's going to be hard to go back to the old way of doing things.

IX. Limitations and Next Steps

This study is observational and has several limitations:

- not every submission links a complete or public repository,
- repository metrics may undercount work done off-platform or in private mirrors,
- year-over-year composition effects (team size, challenge tracks, judging norms, cultural shifts) can confound interpretation,

The next rigorous step is formal change-point and permutation testing around 2023 for large-commit rates, additions-per-file, AI-term frequency, and `.env` leakage rates. A second useful extension is an external control corpus (another student hackathon or similarly timed non-hackathon student repositories).

Next steps would be comparing this TreeHacks data to other hackathons, and potentially to other student projects made outside of the hackathon context. It could also be interesting to do some more formal statistics and permutation testing to see if the changes we see are statistically significant, and to try to control for some of the confounding factors that might be influencing these trends. However, I must admit I'm too overworked to do that analysis before this Wednesday deadline, so I'll leave it for future me.

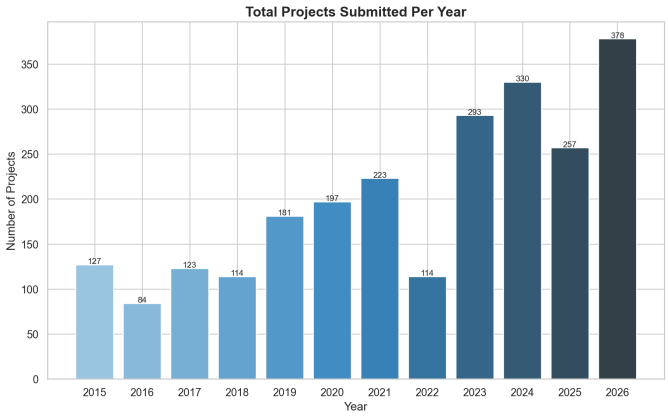


Fig. 1. Project count by year. Recent years show substantially higher participation volume.

X. Conclusion

The evidence does not support the idea that AI simply replaced developers with one-shot code-dumps. Instead, it's more of a structural shift: AI made projects large in scope, more polyglot, and more likely to have bulk code-generation waves, while still showing a fair amount of iterative cleanup and revision.

The most actionable finding is the lack of security concerns. The people committing their API keys are the same ones getting job offers from the top tech companies. Like come on, people, we can do better than this. The vibe-coding pipeline isn't stable enough on its own, it needs our own attention and care to keep it in shape.

Acknowledgments

TreeHacks is an amazing event! I'm grateful to the organizers and volunteers who make it possible each year. I'm also grateful to the many hackers who have now provided endless free API keys for me to abuse in future projects.

Additionally, I should cite that I used Claude Opus 4.6 to help with the DevPost scraping and Github API usage, as well as writing the code that generates the graphs used in this paper. I know that's a bit ironic given the topic of the paper, but I should be transparent.

Finally, thanks to the CS109 staff for the opportunity to do this kind of analysis on a fun topic.

References

[1] Devpost, "TreeHacks 2026," *Devpost*. [Online]. Available: https://treehacks-2026.devpost.com/

[2] A. Karpathy, "There's a new kind of coding I call "vibe coding"," Feb. 2, 2025. [Online]. Available: https://x.com/karpathy/status/1886192184808149383

[3] GitHub, "REST API documentation," *GitHub Docs*. [Online]. Available: https://docs.github.com/en/rest

[4] OpenAI, "Introducing ChatGPT," Nov. 30, 2022. [Online]. Available: https://openai.com/index/chatgpt/

[5] Devpost, "TreeHacks 2022," *Devpost*. [Online]. Available: https://treehacks-2022.devpost.com/

[6] R. McCormick, "vibeathons" [Online]. Available: https://github.com/ryanmccormickdev/vibeathons

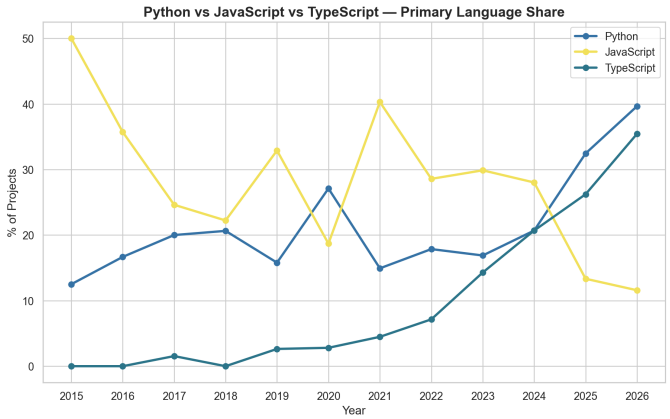


Fig. 2. Share of Python/JavaScript/TypeScript over time. Modern cohorts are less JavaScript-dominant and more Python and TypeScript heavy.

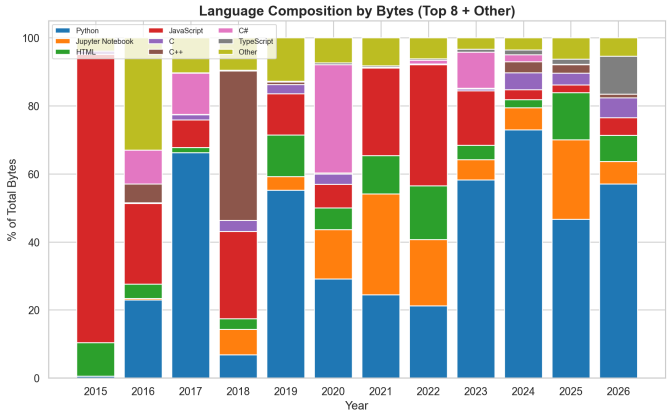
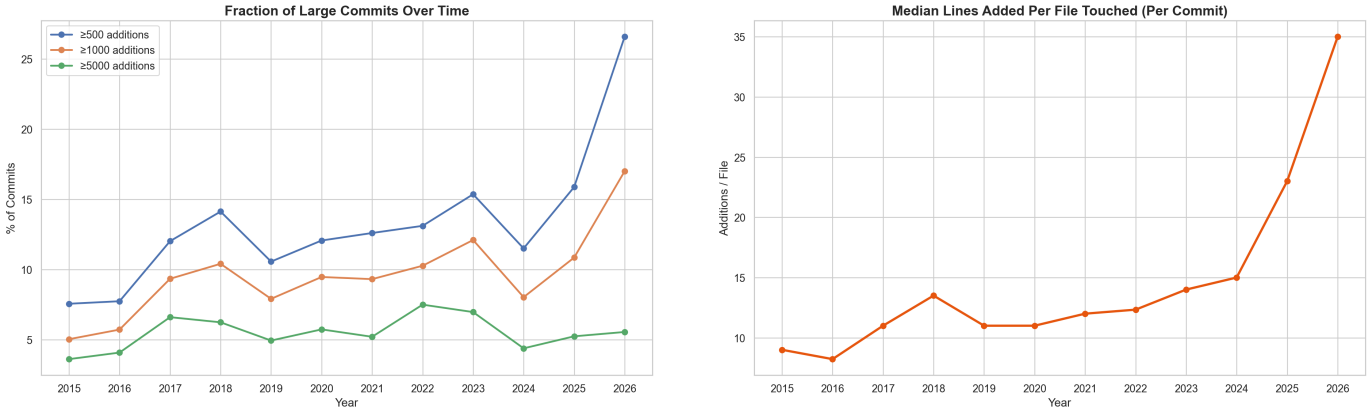


Fig. 3. Stacked language-byte composition by year. Modern cohorts are more polyglot in total volume.

Fig. 4. Multi-metric dashboard of repository and commit behavior. The post-2023 profile sometimes aligns with what we'd expect from vibe-coding, but sometimes is more complex.



(a) Fraction of large commits by threshold (b) Additions per file touched, per commit

Fig. 5. Commit granularity trends. Both views suggest larger code updates during commits.

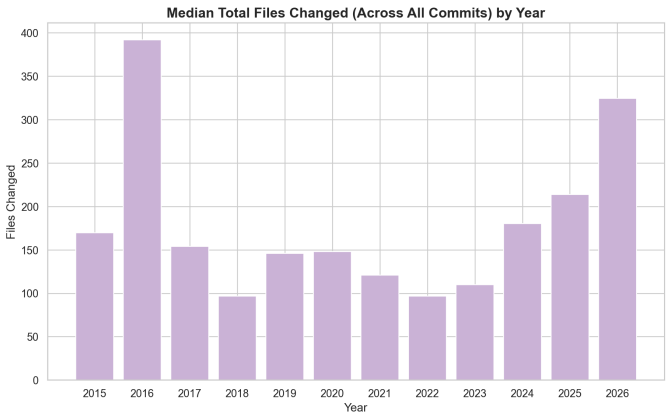


Fig. 6. Total files changed per project. Recent projects touch more code overall.

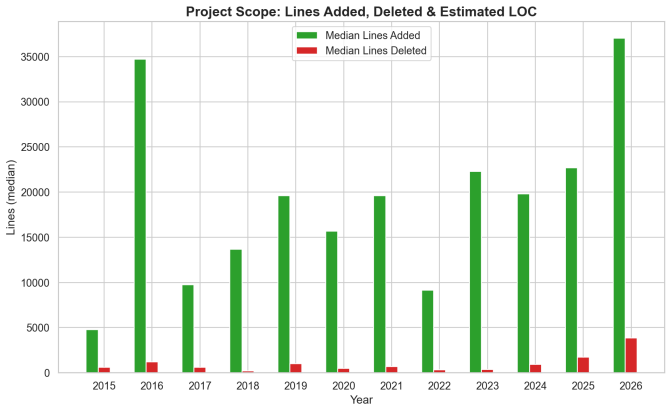


Fig. 7. Lines of code added and deleted per project. Recent projects show a marked increase in code generation.

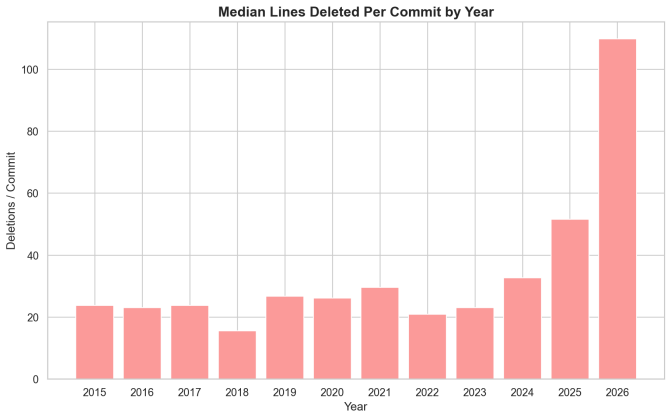


Fig. 8. Deletions per commit by year. Elevated deletion activity suggests continued rework, not only one-shot generation.

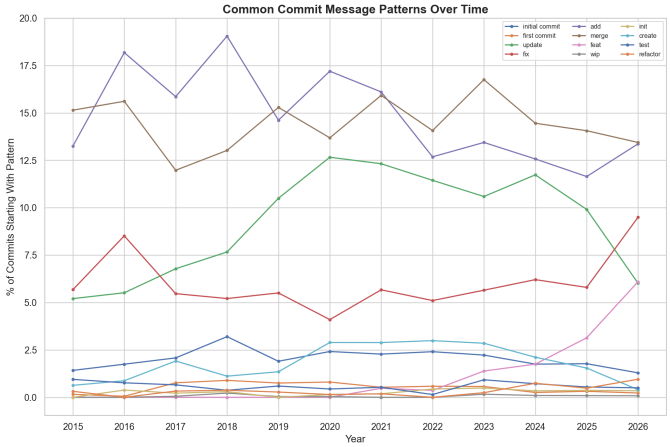


Fig. 9. Frequency of common commit message patterns. The rise of ”fix” and ”feat” suggests more feature-oriented commits, while the decline of ”update” and ”test” might suggest less intentional polishing.

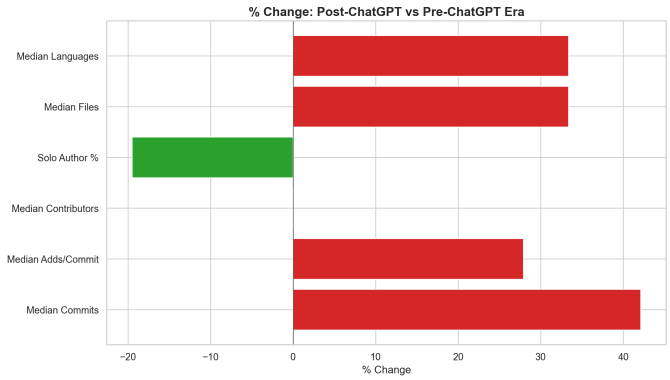


Fig. 10. Percent change for key metrics in pre-vs-post ChatGPT windows. Red indicates a rise in the post-ChatGPT era, green indicates a decline.

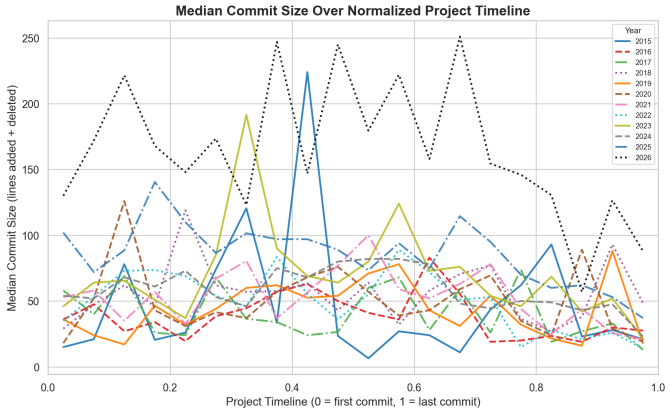


Fig. 11. Commit size distributions relative to project lifetime. Post-2023 projects show higher density of large commits in early stages.

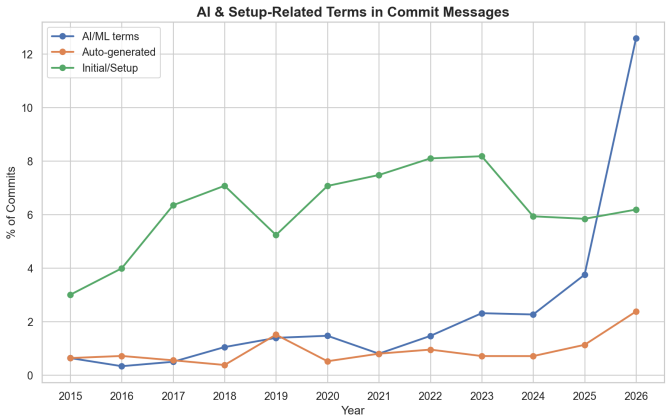
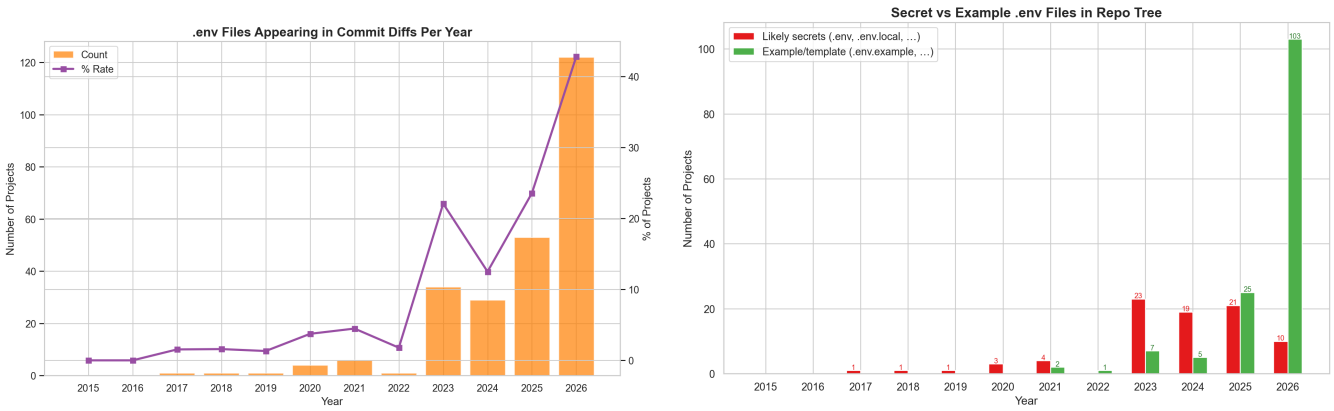


Fig. 12. AI-related terms in commit messages by year. Our definition of AI-related words includes: ”ai”, ”ml”, ”gpt”, ”llm”, ”openai”, ”claude”, ”gemini”, ”copilot”, ”chatgpt”, ”neural”, ”transformer”, ”model”, and ”inference”. Auto-generated terms include: ”auto”, ”generated”, ”scaffold”, ”boilerplate”, ”template”, ”starter”. Initial/setup terms include: ”initial”, ”setup”, ”init”, ”bootstrap”, ”create”, and ”first”.



(a) `.env` presence in commit diffs over time

(b) Secret-like versus example `.env` naming

Fig. 13. Environment-file exposure. Includes files in current tree, as well as ones previously present in commit history. The rise in `.env` presence is concerning, especially given the number of actual secrets that have been exposed.